ELK Stack Training

Module 1

GETTING STARTED

Welcome & Introduction

What You Will Learn

This module is designed to take you from zero to confident with the ELK Stack. You will understand its architecture, install Elasticsearch, and perform your first searches using the REST API.

> Our goal is to give you hands-on, practical knowledge of Elasticsearch fundamentals — from cluster concepts and index mappings to full-text search queries and analyzers.

OBJECTIVES

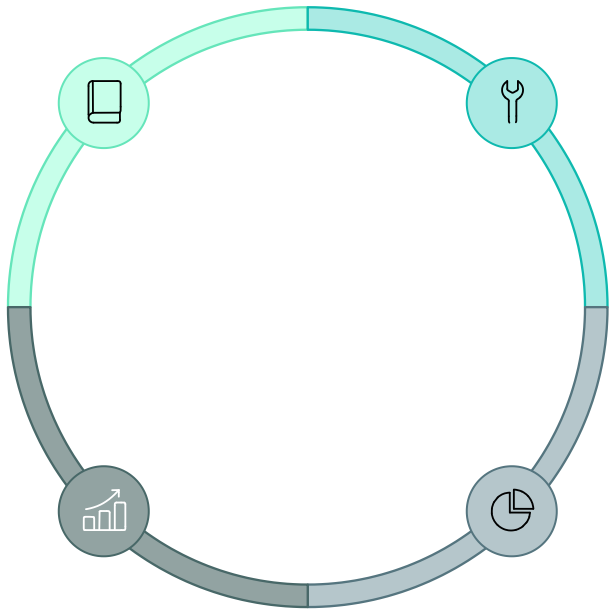
Training Objectives

ELK Architecture

Understand the components of the ELK Stack — Elasticsearch, Logstash, and Kibana — and how they work together in a modern data pipeline.

ELK vs OpenSearch

Compare the ELK Stack with OpenSearch, understanding licensing differences, feature parity, and when to choose one over the other.



Hands-On Setup

Install and configure Elasticsearch via Docker or local setup, and get familiar with core cluster concepts including nodes, shards, and replicas.

REST API Fluency

Perform CRUD operations and execute search queries — match, term, range, and filters — directly against Elasticsearch using the REST API.

SCHEDULE

Module Schedule

Each part builds on the previous one, progressing from conceptual understanding to practical installation and hands-on querying.

Part 1 — ELK Overview

Understand what ELK is, the role of each component, common use cases, and how ELK compares to OpenSearch.

Part 2 — Setup & Config

Install Elasticsearch via Docker or locally, configure elasticsearch.yml, and set up basic access control and security.

Part 3 — ES Basics

Learn index, document, and mapping concepts, then practice CRUD and search queries using analyzers and filters.

PART 1

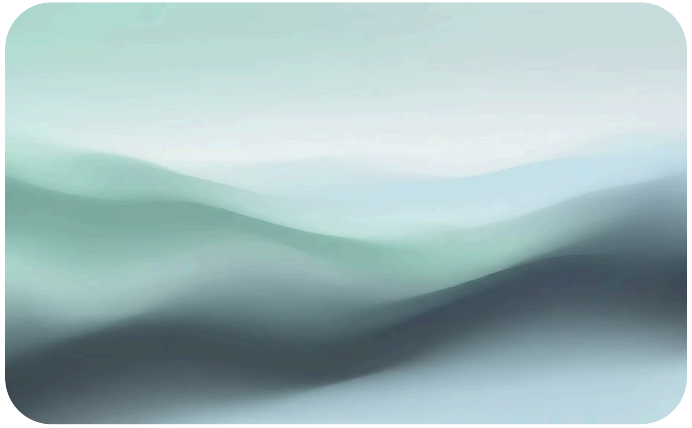
Overview of the ELK Stack

Elasticsearch

A distributed, RESTful search and analytics engine built on Apache Lucene. Stores, indexes, and searches large volumes of data in near real-time.

Logstash

A server-side data processing pipeline that ingests data from multiple sources, transforms it, and sends it to Elasticsearch or other destinations.



Kibana

A web-based UI for visualizing Elasticsearch data. Used to build dashboards, run ad-hoc queries, and monitor infrastructure in real time.

PART 1 — CONTINUED

Use Cases & ELK vs OpenSearch

Common Use Cases

The ELK Stack is widely adopted for log analytics, infrastructure monitoring, application performance management (APM), security event monitoring (SIEM), and full-text search systems.

Log Analytics

Aggregate and analyze logs from servers, containers, and applications to detect errors and anomalies in real time.

Monitoring

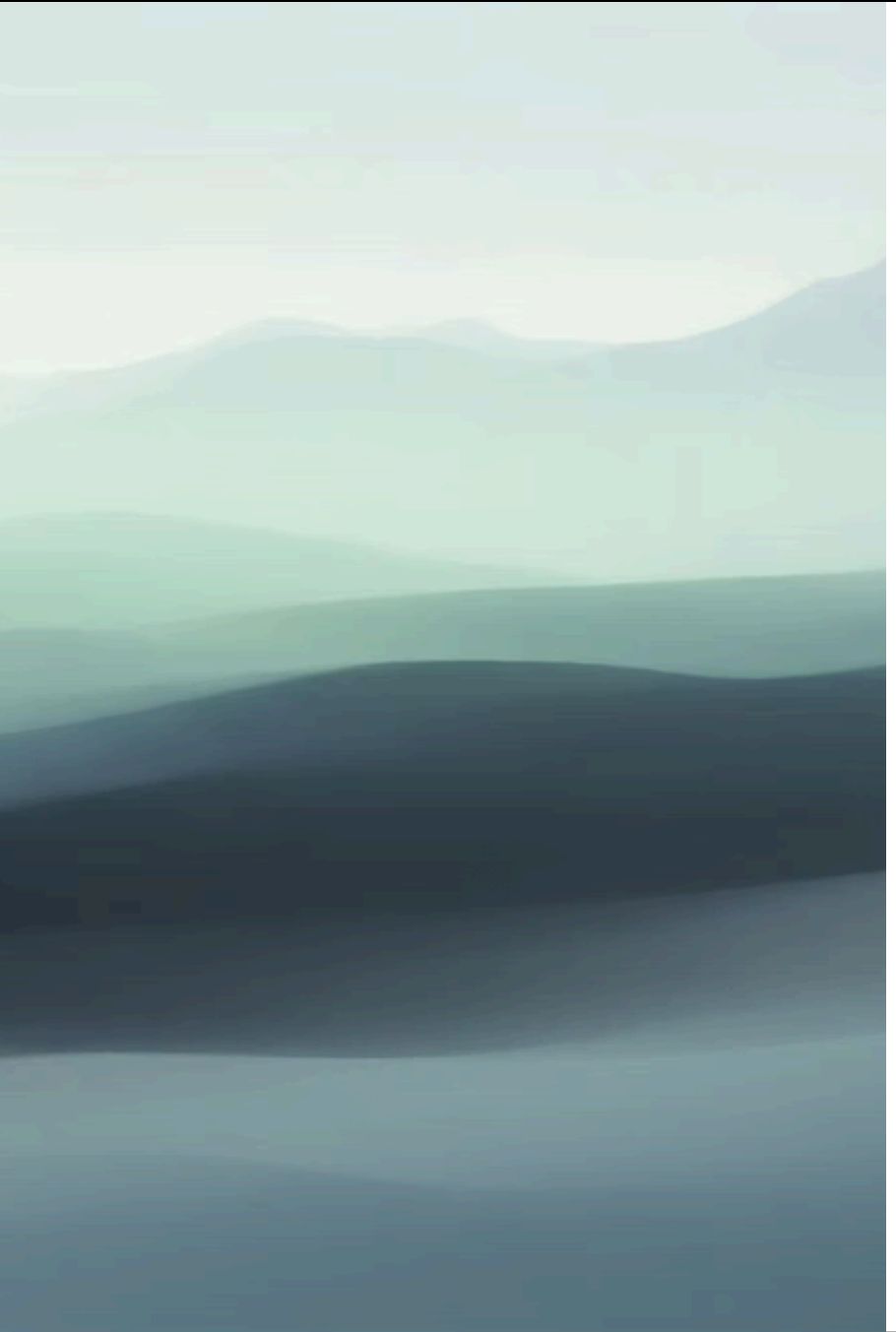
Track system and application metrics, set up alerts, and build operational dashboards using Kibana and Metricbeat.

Search Systems

Power fast, relevant full-text search experiences in applications — from e-commerce product search to document retrieval.

ELK vs OpenSearch

OpenSearch is a community fork of ELK (post-7.10) under Apache 2.0 license. ELK uses the Elastic License 2.0. Both share similar APIs but diverge in features and support.



PART 2

Installing & Configuring Elasticsearch

Docker Setup

Run Elasticsearch quickly using `docker run -p 9200:9200 elasticsearch:8.x`. Ideal for local development and testing without system-level installation.

Cluster Concepts

A cluster contains one or more **nodes**. Data is split into **shards** (primary units of parallelism) and protected via **replicas** (copies for failover and read scaling).

elasticsearch.yml & Security

Key settings include `cluster.name`, `network.host`, and `discovery.seed_hosts`. Enable TLS and built-in user authentication for production deployments.

PART 3

Elasticsearch Basics

Indexes, Documents & Mappings

An **index** is a collection of documents. Each **document** is a JSON object. **Mappings** define field types (text, keyword, date, integer) and control how data is indexed and searched.

CRUD via REST API

Create with `PUT /index/_doc/1`, read with `GET`, update with `POST /_update`, and delete with `DELETE`. All operations are performed over HTTP using JSON payloads.

Search Queries & Analyzers

Use `match` for full-text, `term` for exact values, `range` for numeric or date bounds, and `filter` context for non-scoring queries. Analyzers tokenize text at index and query time.